

Real Time Driver State Detection

Giacomo Reggianini

285040@studenti.unimore.it

Stefano Lugli

244122@studenti.unimore.it

Tommaso Mortara

285139@studenti.unimore.it

University of Modena and Reggio Emilia

Abstract

This paper presents the full development of an AI model of real-time driver's state recognition by incorporating eye state recognition for closed and open states, keypoint estimation, and yawning detection. The main objective was to increase the driver's safety by monitoring their state in real-time using advanced computer vision techniques. We will use diverse datasets to train and validate the models given the robust performance of these tasks.

1. Introduction

In this work, we present an integrated system that utilizes a real-time camera to capture images, which are then analyzed using three distinct models, ultimately combined to provide a complete result. The three models are: Eye State Recognition, Yawning Detector, and Keypoint Estimator.

1. Eye State Recognition: This model is designed to determine whether a person's eyes are open or closed. By analyzing the eye state, we can infer levels of alertness and detect signs of drowsiness or fatigue.
2. Yawning Detector: The yawning detector model identifies when a person is yawning. Yawning is a significant indicator of tiredness and can provide valuable insights into a person's level of vigilance and need for rest.

3. Facial Key Points Detector: This model aims to estimate the interest points on a person's face, providing informations about their posture and movements. By tracking key points, we can monitor behaviors and detect any unusual or concerning patterns.

The combination of these three models allows for a robust and multifaceted approach to monitoring, ensuring that the system can detect a range of indicators related to fatigue, alertness, and overall physical state. The integration of these models into a single system enhances the accuracy and reliability of the monitoring process, providing a powerful tool for real-time safety and vigilance applications.

2. Datasets

In this project, several datasets were collected for training each of the different components of the RTDSD model:

2.1 Eye State Recognition

The dataset used for the training and testing the Eye Detection Model is the MRL Eye Dataset, specifically the mrlEyes_2018_01 collection. [1] The MRL Eye Dataset includes a substantial number of infrared images captured under diverse lighting conditions and with different devices, ensuring robustness and generalizability of trained models. The dataset encompasses 84,898 images from 37 subjects, annotated with key attributes

such as gender, presence of glasses, eye state (open or closed), reflections, lighting conditions, and the type of sensor used.

To optimize the supervised training process, the dataset was pre-processed to divide images into two distinct categories: 'open_eyes' and 'closed_eyes'. This organization enhances the efficiency of the training phase, allowing for more precise model development and testing.

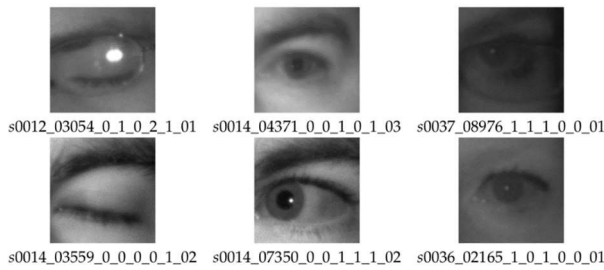


Figure 1: An examples of image annotations of the proposed dataset

2.2 Keypoint Estimation

The dataset employed to perform the training and testing stage of the Facial Keypoint Detection model has gone through a major change. During the first stage of coding, the 68 facial key point detection dataset [2] was used, but, after considerations about the quality of the predictions of the computed model, we've decided to use a new dataset that uses the same data, but all the images are cropped around the face and the key points are modified accordingly [3].

These are a processed version of the [YouTube Faces Dataset](#), that basically contained short videos of celebrities that are publicly available and were downloaded from YouTube. There are multiple videos of each celebrity (up to 6 videos per celebrity).

The employed dataset consists of 3438 images for training and 766 for testing. The dataset has been completed with annotations that consist in two csv files, training.csv and test.csv, that contain in the first column the name of the file they refer to and the corresponding line is full of the 68 coordinates (x, y) of the actual points in the image.

2.3 Yawning detection

The most used dataset for Yawning detection is the YawDD [4] distributed by IEEE. It is composed of several videos, from two different angles (front and side) portraying people Yawning, talking, or simply driving. Multiple actors of different sexes and ethnicities were employed with different light conditions, to make sure there is no bias in the result. The dataset was partially labeled (only the side videos had titles telling if it would contain someone yawning or not), and even where there was someone yawning (supposedly) most of the video would picture a person just driving, and only occasionally yawning.

Therefore, before training, we needed to divide all the videos into frame sequences and tag which frame would contain someone yawning or not.

When preprocessing the videos to extract the frames, where possible, faces were detected using haar cascade models, and cut out of the picture to build a dataset made mostly of just faces, cutting the noise from the surrounding.

Pre-processing of each dataset was done to ensure the models developed would have a general ability. Much emphasis was put on data cleaning, normalization, and augmentation to show various environments and driver behaviors.

3 Approach

The development of the Real-Time Driver State Detection system has been elaborated as the development of 3 specialized AI models for each detection task.

3.1 Eye State Recognition

The goal of this model is to determine whether a person's eyes are open or closed once a face has been detected. As previously mentioned, a pre-processing phase was necessary to correctly divide the dataset so that the model could accurately learn which eyes are open and which

are closed. To achieve this, the MRL Eye Dataset was segregated into two categories: 'open_eyes' and 'closed_eyes', which facilitated the supervised training process.

For data augmentation, we utilized the 'ImageDataGenerator' from TensorFlow's Keras API [5] to enhance the diversity of the training dataset. The augmentation included rescaling the images, applying rotations, shear transformations, zooming, shifting, and horizontal flipping. This augmentation helps the model generalize better to new, unseen data by introducing variability into the training process.

The dataset was split into training, validation, and test sets. The training and validation sets were created using an 80/20 split, while the test set was prepared separately. This ensured that the model's performance could be evaluated on a completely independent set of images.

For the model architecture, we employed MobileNetV2, a lightweight and efficient convolutional neural network pre-trained on ImageNet [6]. Only the last 50 layers of the MobileNetV2 base model were unfrozen to be trainable, which reduces the number of parameters and focuses training on the most relevant features for our specific dataset, and additional custom layers were added to tailor the network to our specific task. These layers included a flattening layer, a dense layer with 64 units and ReLU activation (along with L2 regularization to prevent overfitting), batch normalization, dropout for regularization, and a final dense layer with a softmax activation to classify the eye state into two categories (open or closed).

To optimize the training process, we incorporated several callbacks:

1. **ModelCheckpoint** to save the best model based on validation loss.
2. **EarlyStopping** to halt training if the validation loss did not improve for a specified number of epochs, thereby preventing overfitting.

3. **ReduceLROnPlateau** to reduce the learning rate if the validation loss plateaued, ensuring a more refined convergence.

The model was compiled with the Adam optimizer, a categorical cross-entropy loss function, and accuracy as a metric. The training process involved fitting the model on the augmented dataset, with the model's performance being evaluated at each epoch to ensure optimal accuracy and loss.

This approach ensured a robust and efficient eye state recognition model, capable of accurately determining whether eyes are open or closed in various conditions.



Figure 2a/2b: Examples of the model working on a real-time camera.

Model Evaluation

During the development of our eye state recognition model, we experimented with several different neural network architectures to identify the most effective approach. These included various deep learning models known for their strong performance in image classification tasks, such as MobileNetV2, ResNet50, MobileNetV2 with some layer off, ResNet50 with some layer off, InceptionV3 and an HandCrafted CNN made by 16 Convolutional layer.

Despite always achieving pretty high accuracy, some models exhibited significant errors during the testing phase. This discrepancy suggests that these models were likely overfitting the training data, capturing noise and specific patterns that did not generalize well to unseen data.

To mitigate this issue, we emphasized the use of techniques such as regularization, dropout, and early stopping, which helped reduce overfitting to some extent.

The results obtained from these experiments are summarized below, highlighting the accuracy and the loss achieved on the testing sets for each model.

	MobileNetV2LowParam	ResNet LowParam
Test Loss	0.0899211	0.0921197
Test Accuracy	0.9786331	0.9742279
	MobileNetV2	InceptionV3
Test Loss	0.1429470	0.1254375
Test Accuracy	0.9640510	0.9538975
	ResNet	HandCrafted CNN
Test Loss	0.1034886	0.1136021
Test Accuracy	0.9682207	0.9242279

Figure 3: Comparison between scores obtained by all the differents model trained

3.2 Keypoint Detection

Task

Our task, in the creation of the facial key points detection model, is self-explained in the name: we needed to train a model to understand and predict the position of the facial landmarks in a given image in the form of key points. Aiming to resolve this problem, we created a supervised learning model, whose structure and training will be explained later.

Data Processing

In the FaceKeypointDataset class, inside the `__getitem__` method, we've defined the preprocessing operations to apply.

We divide the preprocessing into two main stages, one for the images and one for the key points.

The image preprocessing starts by reading the image corresponding to the name in the first column of the csv file, whose path to the correct directory is passed as a parameter when we call the constructor method and then it converts the entire image from BGR to its RGB correspondent. The images given in the dataset are all of different dimensions, so we resize them to a selected dimension, 224x224, to conform to the dimensions that best suit the usage of the resnet50 model, but saving the original dimensions that will be useful later for key points preprocessing. The next operations are the normalization of the pixel values of the image to the range $[0, 1]$, and the transposition of the image channels from (height, width, channels) to (channels, height, width), which is the format expected by PyTorch. Finally the images are converted from numpy arrays to PyTorch tensors.

The key point preprocessing, after extracting the values from the csv file, converts them to a numpy array and resizes it into pairs of (x, y) coordinates. The next operation consists of rescaling the key points according to the resized image dimensions. This ensures that the key points correspond to the resized image rather than the original image. We finally convert the numpy arrays to PyTorch

tensors.

These operations are iterated for every line in the csv file passed as a parameter.

The method finally returns the image and corresponding key points as a single object.

Architecture

In order to implement a network able to predict the 68 facial key points on a given image of a face, we used the architecture of ResNet50.

ResNet-50 is a convolutional neural network (CNN) architecture developed by Microsoft Research Asia. This ResNet architecture has 50 layers, and is one of the deep learning models most widely used in various computer vision tasks such as image classification, object detection, and segmentation. ResNet-50 uses a residual learning approach, which enables the neural network to learn more complex features and avoid the so-called "degradation problem" as the number of layers increases.[6]

ResNet-50 incorporates 50 bottleneck residual blocks, arranged in a stacked manner. The early layers of the network feature conventional convolutional and pooling layers to preprocess the image before it undergoes further processing by the residual blocks. Ultimately, fully connected layers positioned at the pinnacle of the structure utilize the refined data to categorize the image with precision. In our project, the last fully connected layer has been modified to reshape the output in the optimal shape, that is an array of 136 values: this object can be easily reshaped into a (68, 2) tensor to use as the coordinates of the predicted key points.

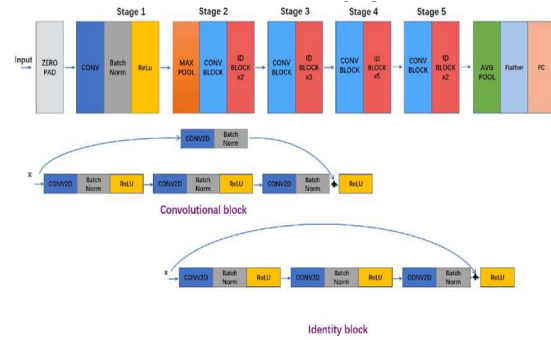


Figure 4: ResNet50 Standard Form

In addition to the ResNet50 capabilities, we've used also the pretrained weights available by default with the usage of this architecture, that are ResNet50_Weights.IMAGENET1K_V2. [7] The purpose of this is to better the final results: their accuracy and precision.

Training

We've seen how we've treated and processed the given data from the dataset, and which model has been selected to perform training. We've also analyzed the fine tuning operations on its standard form, applied to yield better results: now we're going to see how we combined these two fundamental pieces for training a performing model that predicts the 68 facial key points.

Before beginning the training process, a couple of parameters must be selected: we have chosen the Adam optimizer, with a learning rate of 0.0001, and, as loss function, the SmoothL1Loss:

$$\text{loss}(x, y) = \begin{cases} 0.5(x - y)^2, & \text{if } |x - y| < 1 \\ |x - y| - 0.5, & \text{otherwise} \end{cases}$$

Figure 5: Smooth Loss L1 Formula

After this initialization of the necessary parameters, we proceed to the training and validation stages. The custom 'fit' method, after setting the model to the training mode, iterates over the batches of data, the necessary operations, like the key points flattening, to convert them to a one-dimensional array, the forward pass, the loss computation, and others.

The ‘validation’ method is fairly similar, but the model is set in evaluation mode.

After a 100 epochs training stage, we can see the loss curves both for training and validation:

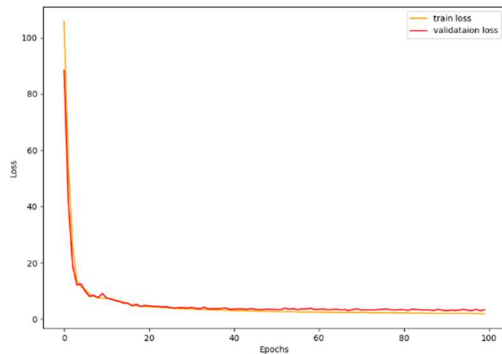


Figure 6: Loss Curves During Training

We add a selection of the predictions in evaluation on a test image during the training epochs to see its progression:

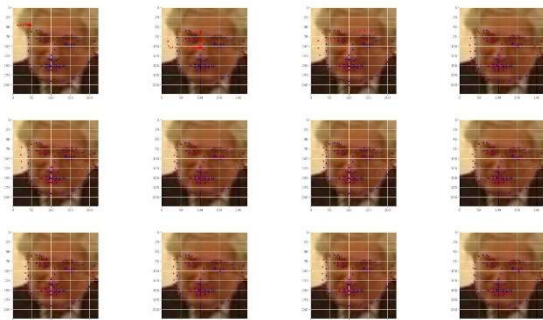


Figure 7: Key Point Detection Progression During Training

After having saved the model, a quick code to test our product at runtime on the available camera returns the following results:



Figure 8: Real-Time Inference of The Model

3.3 Yawning Detection

The objective of this model is to detect if a driver is yawning, maybe the most important indicator when comes to “predicting” drowsiness at its first stages.

Data Preparation and Augmentation

To ensure the model can generalize well to new data, despite having already an extensive dataset, an ImageDataGenerator from TensorFlow's Keras API is used in similar way to Eyes state detection.

Model Architecture

The base model used is, MobileNet, a lightweight and efficient convolutional neural network pre-trained on the ImageNet dataset. This choice is driven by MobileNet’s balance between performance and computational efficiency, making it ideal for embedded applications.

1. **Transfer Learning Setup:** The MobileNet model is loaded without the top layers, leveraging pre-trained weights to extract robust features from images. All layers of the base model are initially frozen to retain the pre-trained weights and prevent their modification during the initial training phase.
2. **Custom Layers:** Global Average Pooling is added to reduce the spatial dimensions of the feature maps, leading to a more compact representation. A fully connected layer with 128 units and ReLU activation to introduce non-linearity and learn complex patterns. A 50% dropout rate is applied to prevent overfitting by randomly dropping units during training.

A dense layer with 2 units and softmax activation for binary classification (yawning vs. not yawning) is used as output layer.

Training Process

1. **Class Weights:** To handle class imbalance, class weights are computed

and applied during training, ensuring the model pays more attention to underrepresented classes (yawning).

2. **EarlyStopping:** Monitors the validation loss and stops training when it stops improving, helping prevent overfitting.
3. **Initial Training:** The model is compiled with the Adam optimizer, categorical cross-entropy loss, and accuracy as the metric. The initial training phase involves fitting the model with frozen base layers, allowing the custom layers to adapt to the specific task of yawn detection.
4. **Fine-Tuning:** The last 10 layers of the base model are unfrozen, allowing fine-tuning of the pre-trained weights for better feature extraction specific to yawn detection. The model is recompiled with a lower learning rate to make smaller, more refined updates to the weights.

Model Evaluation

- **Accuracy and Loss:** The model is evaluated on the validation set to determine accuracy and loss.
- **Classification Report:** Provides detailed metrics such as precision, recall, and F1 score for each class.
- **Confusion Matrix:** Visualizes the performance by showing the true vs. predicted classifications, aiding in identifying misclassifications.
- **Precision, Recall, F1 Score:** Key metrics are calculated to assess the model's performance comprehensively.

The resulting scores did not look promising at all, but when the model was tested on real data (e.g. on real time webcam) the detection was mostly good, even with multiple targets at the same time.

The overall results during testing phase were the following:

Table 1 testing metrics

Precision	0.5013620029713767
Recall	0.5033452167875954
F1 Score	0.4849518177461815

With the following confusion matrix:

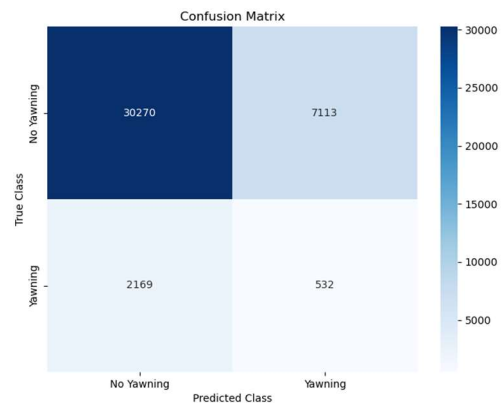
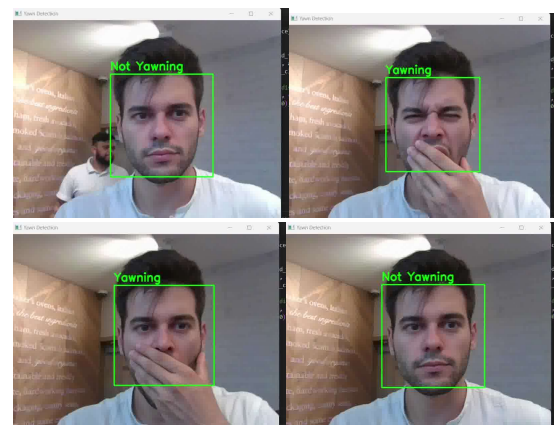


Figure 9: Confusion matrix during testing phase

While on actual data, when camera was positioned with the expected angle (e.g. front or side) and decent enough lighting was applied (for low light situations a fully black and white infrared camera could be suitable), the error was way lower and yawning class is mostly detected correctly, both with and without hands in front of the mouth.



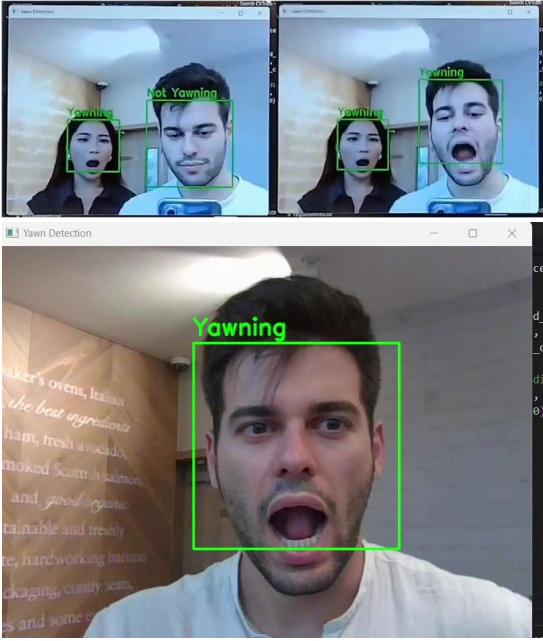


Figure 10a/b/c: Frames taken from the first real world test

Conclusion

This model leverages transfer learning with MobileNet, data augmentation, and fine-tuning techniques to achieve robust yawn detection. The use of class weights, early stopping, and dropout layers further enhances its performance, making it suitable for implementation in an embedded device for driver alertness monitoring.

4 Union of the models

The system is designed to operate in real-time, leveraging the multiple deep learning models for the three different tasks. The architectural design is modular, with each module responsible for a specific aspect of driver monitoring.

The implementation heavily relies on multithreading to ensure the system operates in real-time. The use of threads allows the system to capture frames from the webcam, process these frames, and update the display concurrently. This design choice is critical for

maintaining low latency and high throughput, essential for a real-time monitoring system.

Each model processing runs on a different thread, with one extra thread used for final state computation.

To ensure the models receive input in the correct format, the system includes preprocessing steps such as resizing, normalizing, and expanding the dimensions of the input frames.

The system employs a circular buffer to store recent log entries, ensuring that memory usage remains constant and manageable. Logs are maintained in both plain text and JSON formats, providing flexibility for further analysis and debugging.



Figure 11: testing models union. On the left, the final computed driver state. On the right, the single models results are displayed over the video input.

The system computes various metrics based on the logs, such as the rate of yawning, eye state transitions, and facial keypoints geometries. These metrics are used to infer the driver's state and display appropriate alerts. This functionality is essential for providing actionable feedback to the driver.

1. Yawn Rate: Yawn rate is computed as the number of yawns per unit of time (e.g., yawns per 15 seconds). If the yawn rate exceeds a certain threshold, it is an indicator of driver drowsiness. The strength is that it directly correlates with fatigue and can be a reliable indicator of drowsiness.

2. Eye Closure Frequency and Duration: The frequency and duration of eye closures are tracked, measuring how often and how long the driver's eyes remain closed. A threshold is computed based on the proportion of time the eyes are closed over a specific period. Abnormally long or frequent blinks can indicate drowsiness.

3. Facial Key points and Geometric Analysis: Different metrics may be applied with these informations. For now, measuring the distances and angles between facial key points, such as the angle formed by the eye or the relative distance between chin and nose may tell us with what angle the head is moved away from a given starting point, letting us understand if the driver is either falling asleep or actively scanning the streets.

Conclusion

Modularity and models independence allows easy maintenance and ease of update, while the use of textual logs for decision making permits the application of different approaches. For instance, it would be quite easy to create a new dataset made of multiple tagged video, showing people driving in drowsy and not drowsy attitudes, then applying this script, create a log for each video and train a lightweight machine learning algorithm for understanding such sequences of log, turning the complexity of video analysis into straightforward classification tasks.

5 Future Developments

The developed product can be further improved in a few different directions:

- CNN Architecture Improvement: Enhance the architecture of convolutional neural networks to increase the precision and reliability of driver state detection, ensuring the model adapts better to various real-world scenarios. Some improvements may also be considered, like coherent input sizes for the different models and usage of related frameworks,

allowing for faster preprocessing at runtime and simpler development.

- Better exploiting of the current result: the best feature of our approach is the capability of turning some complex (like a sequence of events in a video) in a simple textual log, which can be analyzed in a very easy and straightforward way. For example, this models combination may be used to for analyzing tagged video of drowsy/not drowsy people, extract a tagged log for each one of the video, and teach a simple classifier to recognize fixed-length sequences of events that may be bringing an user to a risky situation.
- Usage of LLMs: taking advantage of the human readable output log, we tried to ask to LLAMA 3 [9] if a sequence of logs may or may not contain a drowsy attitude. Giving firm instructions about getting an exact reply (yes or no), the answer would be easily parseable. In our tests, the answer was almost always correct, without the need to adapt anything in our output. The ability of turning a video in a lightweight text that can also be posted easily to some complex LLM APIs, opens up to the scenario of creating semi-real-time monitoring systems that could give instructions to long distance drivers (like truck drivers) on what they should be doing, or update navigational systems making them aware of the driver's conditions so that the trip could be adjusted counting extra stops to preserve passengers' safety.
- A mix of the previous two points: once made sure the models applied were good enough at their simple tasks, we may start tagging datasets through a two-stage approach: first logging the events in each video, then exploiting the LLM comprehension capability to tag the content of the video. Such approach may not be limited to driver drowsiness, but to quite a lot of scenarios given the right set of models, turning the task of dataset labeling and supervised learning into an unsupervised one.

In general, the ability of translating the complex semantics of a video into simple data/text sequences can allow developers to create more complex systems without being stuck in hardware/cost constraints.

6 Conclusion

The world of Computer Vision and Artificial Intelligence is finding some very useful applications in the automotive world, particularly in the world of autonomous driving or driver safety.

With the described work, our team tried to implement a real-time driver state detection project that fits in this field, with pretty good results.

The final project does work with good accuracy and an acceptable frame rate on a controlled environment: we ran the scripts on our laptop with the built-in webcam, filming our faces with good lighting.

The project runs at acceptable frame rates and is resistant to some variation of light and scale, so its usage in real life is possible, but with limitations. This being an academic project, it presents clear restrictions, but we already analyzed some possible future developments through which we can achieve better precision and invariance to external factors.

Bibliography :

[1] :

<http://mrl.cs.vsb.cz/eyedataset>

[2] :

<https://www.kaggle.com/datasets/selfishgene/youtube-faces-with-facial-keypoints>

[3] :

<https://www.kaggle.com/datasets/sovitath/cropped-face-keypoint-dataset-68-landmarks?resource=download>

[4] :

[YawDD: Yawning Detection Dataset | IEEE DataPort \(ieee-dataport.org\)](http://www.yawningface.org/)

[5] :

https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator

[6] :

K. Dong, C. Zhou, Y. Ruan and Y. Li, "MobileNetV2 Model for Image Classification," 2020 2nd International Conference on Information Technology and Computer Application (ITCA), Guangzhou, China, 2020.

[7]:

https://www.researchgate.net/publication/370531243_Implementation_of_ResNet-50_on_End-to-End_Object_Detection_DETR_on_Objects , pag. 1090

[8]:

<https://pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html>

[9]:

[Welcome Llama 3 - Meta's new open LLM \(huggingface.co\)](https://huggingface.co/Llama-3)